

Automatic Estimation of FM Synthesis Parameters

Anonymous Authors

Abstract—FM synthesis is one of the most renowned techniques for sound synthesis, and although it is an old method, it remains widely used today. It is simple, computationally inexpensive, and capable of producing high-quality results. However, the successful use of this type of audio synthesis presents a significant challenge: selecting the correct parameters for the synthesizer. This process is not precise, and there is no fully deterministic procedure to follow. Musicians are usually guided by intuition and an empirical trial-and-error process. To revive this technique and leverage its benefits, the use of Machine Learning models appears to be a very promising option, as they can help musicians achieve convincing results, imitating real instruments in a short time and at a low cost, or even creating totally unpredictable timbres and effects. Therefore, this work explores this approach by creating a pipeline capable not only of training a model to predict the appropriate parameters that enable an FM synthesizer to resynthesize original audio samples (such as acoustic instruments), but also of exploring the synthesizer’s parameter space broadly, allowing it to reproduce any arbitrary desired sound. The proposed approach proved to be efficient for both objectives: the developed model was able to approximate unseen sound samples (of real instruments), and the error metrics converged to satisfactory values even when using an exploratory dataset generation process (a synthetic dataset specifically constructed to explore the FM synthesizer’s capabilities).

I. INTRODUCTION

Musical expression is one of humanity’s earliest experiences and one of the most valued forms of art. This art form has been cultivated for centuries. The Chinese, three thousand years before Christ, had already developed complex systems of musical theory, and Greek culture not only cultivated its study but also associated it with their religion, assigning divine characteristics to protect and honor the music itself [?].

Although music is a subjective form of human knowledge, the recorded music industry today generates approximately US\$29 billion [?] annually, and thus its value ceases to be purely subjective.

It is precisely in this context that the artifice of synthesized sound appears in history. It emerged after the first electronic instruments and, due to the periodic nature of sound, the earliest synthesizers were composed of simple analog oscillators that could be manually controlled through operations such as summing and filtering [?].

In the 1980s, with the release of the Yamaha DX7 synthesizer, the history of music synthesis took a major turn. The Yamaha DX7 marked the first major success of a revolutionary technique developed by John M. Chowning at Stanford University and later popularized by Yamaha [?].

This event represented a new milestone in sound synthesis, not only because of its popularity, but also because it made the reproduction of both realistic instrument simulations and entirely new kinds of sounds easily accessible to musicians and

composers of all levels. In fact, the success of FM synthesis was so great that the technique remains in use to this day [?].

Thus, the primary goal of the present work is to guide a frequency modulation (FM) synthesizer to reproduce instrument sound samples, while also developing a process that fully explores the expressive capabilities of the FM synthesizer itself, approximating the re-synthesis of arbitrary input sounds.

This approach aligns with one of the key historical advantages of FM synthesizers: their ability to create sounds that cannot be produced by traditional acoustic instruments.

Although FM synthesis is considered an old technique compared to approaches such as wavetable synthesis or GAN-based sound generation, it remains widely used within today’s complex musical ecosystem due to its advantages: it is easy to use, well-suited for experimenting with new and even unpredictable sounds (such as spatial effects), and easily adaptable to real-time composition.

So, researchers continue to focus on improving the control of traditional FM synthesis parameters (which, when done manually, can be a tedious empirical process). For instance, [?] implemented a neural network to predict FM synthesizer parameters using the frequency-domain sound representation as input. Similarly, [?] proposed an interesting TCN-based approach that couples the FM synthesizer directly into the network training process.

The current work, however, initially explored the use of a Convolutional Neural Network (CNN) to achieve its objectives, given that CNNs have been shown to be effective feature extractors for audio signals [?]. The approach subsequently evolved into a more complex architecture that simultaneously uses raw audio samples, frequency-domain representations, and Log-Mel spectrograms as model inputs, while progressively predicting different families of FM synthesizer parameters.

Furthermore, rather than constituting a simple re-synthesis exercise, the present work aims to develop a framework that enables the model to acquire a broad understanding of the entire parameter space of the FM synthesizer under consideration. The central hypothesis is that, if the model can learn this parameter space comprehensively, it can subsequently be used to reproduce any sound that lies within the synthesizer’s capabilities.

This would make the workflow of a potential end user, such as a musician or composer, substantially more efficient when searching for a desired sound, while eliminating the need for specialized dataset preparation and separate model training for each target timbre or voice.

The remainder of this paper is organized as follows. Section ?? presents the fundamental concepts related to music. Section ?? discusses the related work. Section ?? describes

the details of the proposed approach and the experimental setup. Section ?? presents and analyzes the results. Finally, Section ?? provides the conclusions of this work and outlines directions for future research.

II. MUSICAL CONCEPTS

Although this work is grounded in established machine learning methods, its application relies on fundamental notions from music and sound analysis. This section summarizes the concepts most relevant to the study.

A. Musical Computation

Musical computation is an interdisciplinary field dedicated to the digital processing of musical information. While a comprehensive overview is beyond the scope of this article, it is necessary to outline the basic physical and computational elements of sound.

Audio is a mechanical pressure wave that can be converted into an electrical signal by a microphone. As a wave phenomenon, sound is characterized by:

- **Pitch:** The fundamental frequency determining perceived height.
- **Duration:** The temporal extent of an audible event.
- **Loudness:** The perceived intensity of the waveform, linked to amplitude.
- **Timbre:** The spectral “color” of a sound, defined by its harmonic structure.

Acoustic instruments rarely produce pure sinusoids: multiple resonant components vibrate simultaneously, shaped by the instrument’s material and geometry. This superposition of oscillations results in distinct timbres, which are typically analyzed by transforming the time-domain signal into the frequency domain via the Fourier Transform [?].

B. Audio digitization

Digital audio requires the discretization of the analog signal. Sampling [?] records amplitude values at regular time intervals. According to the Nyquist–Shannon theorem, the sampling rate must exceed twice the highest frequency to be represented. Although human hearing extends up to approximately 20 kHz, music analysis tasks commonly use sampling rates around 16 kHz, which adequately capture the spectral content of most instruments.

C. Sound synthesis

Three mathematically grounded synthesis techniques are especially relevant:

- **Additive synthesis:** Constructs sound by summing sinusoidal components.
- **Subtractive synthesis:** Starts from a harmonically rich signal and applies filters to shape the spectrum.
- **FM synthesis:** Produces complex spectra through frequency modulation between oscillators.

FM Synthesis, introduced by Chowning [?], generates sidebands around a carrier frequency, enabling the creation of rich timbres with relatively few parameters. Its efficiency and

flexibility contributed to the success of the Yamaha DX7 synthesizer [?], [?].

D. Audio features and comparison

Comparing audio signals in a perceptually meaningful way is challenging and computationally demanding. Standard methods [?] include:

- **FFT Distance:** Euclidean distance between magnitude spectra obtained via the Fast Fourier Transform.
- **STFT Distance:** Extension of the previous metric using short, overlapping time windows.
- **Log-Mel Spectrogram Distance:** A perceptually motivated version computed on the Mel-frequency scale [?], [?].

Envelope-based metrics may also be used, but since this study focuses on timbral similarity, envelope comparison falls outside its scope.

III. RELATED WORK

It is not the first work to explore using a neural network to predict appropriate parameters for a sound synthesizer. However, the main innovation here is the choice of CNNs to analyze the sound to be imitated. The main reasons are presented below:

- The ability of CNNs to process raw audio inputs.
- The potential of CNNs to capture the temporal structure of sound samples.
- The near real-time performance of CNNs, after training, which can be an advantage for future practical applications.

However, the following works were consulted to support this research and to guide the methodological comparison. The results, in fact, are difficult to compare due to the intrinsically different measurement scales:

[?] proposed the re-synthesis of instrumental sounds using machine learning models combined with an FM synthesizer. However, [?] did not use CNNs or the raw signal. Instead, he used the STFT spectrogram as input (i.e., a frequency-domain representation of the signal), which may simplify the learning process but does not fully exploit the temporal information in the audio.

[?] introduced DDX7, a differentiable FM synthesizer designed for the re-synthesis of musical instrument sounds. The strength of this work lies in its use of a Temporal Convolutional Network (TCN). However, their approach differs from the present work in the target values chosen for the model. DDX7 operates directly on the Mel-spectrum representation, integrating it into the loss function. This was remarkable, as it provided the neural network with a loss function aligned with one of the most common audio comparison metrics. At the same time, this approach requires the synthesizer to be included in the training loop, which is more computationally expensive in terms of both memory and processing. Furthermore, compared with the present work, this research emphasizes strong generalization by generating a wide range of

parameter combinations through controlled random selection of FM parameters, as explained in the next section.

Finally, [?] presented a comprehensive survey on 1D convolutional neural networks (CNNs) and their applications, highlighting their effectiveness in sequential data such as audio signals. Although this work does not focus on the same purpose (namely, the re-synthesis of audio samples), it explores various use cases for one-dimensional convolutional neural networks, including audio signal processing. This survey is an important reference that validates the applicability of CNNs to audio signals and their ability to capture the temporal structure of sounds.

IV. METHODOLOGY

This section introduces our proposed approach of using AI to guide an FM synthesizer in simulating arbitrary instrument or arbitrary desired sound. More specifically, we propose a Convolutional Neural Network (CNN) to analyze a sound signal sample and predict the correct parameters for an FM synthesizer to re-synthesize the “same” timbre, enabling the synthesizer to simulate the desired “voice” playing any song. Since this idea is still quite abstract and broad, we consider the following steps:

- 1) **Implementing FM synthesizer:** Implement a simple FM synthesizer, which can later be used both to generate the dataset and to test the ability to re-synthesize a sound.
- 2) **Generating the dataset:** Generate a synthetic audio dataset composed of random samples, each created by randomly selecting FM synthesizer parameters. This essentially records tuples of (parameter values, audio samples) and explores the parameter space at a large scale.
- 3) **Training the CNN:** Train a CNN to predict the correct parameters for the same FM synthesizer, given only an audio sample.
- 4) **Evaluating the Model:** Evaluate the model’s results using the NSynth dataset to test its generalization capabilities.

Frequency Modulation Synthesis was chosen because it is one of the most common and effective methods for generating audio samples, owing to its simplicity and low computational cost.

This technique became extremely popular in the 1980s with the release of the Yamaha DX7 synthesizer. By the 1990s, it had become a standard for many commercial synthesizers and remains in use today in electronic music production [?]. However, the effective use of this technique requires advanced knowledge to select the correct parameters for achieving good results, which can involve thousands of parameters.

Therefore, the ability to simplify parameter selection provides a strong justification for this study, and AI models appear to be an affordable way to learn the right patterns for controlling an FM synthesizer. The basic idea is that a model can be produced to guide an FM synthesizer in mimicking

any sound, thereby optimizing the effort to compose electronic music.

Furthermore, in terms of computational cost, using an AI model to predict parameters and guide an FM synthesizer is much more effective than applying a Generative Neural Network to generate an audio. Additionally, a synthesizer is particularly well-suited for real-time composition and electronic musician performance.

Regarding the configuration, all experiments were conducted in an environment with Python 3.11.2, TensorFlow 2.19.0, Keras 3.11.1, Nvidia-cuda 12.5.82, Nvidia-cudnn 3.0.75, Nvidia Driver 535.247.01, CUDA 12.2, and a GPU NVIDIA GeForce GTX 1650 (4GB memory).

A. Implementing FM synthesizer

There are several well-known FM synthesizers on the market, but most are not open-source. Additionally, commercial synthesizers often have a vast number of parameters that can be used to control the audio production process. For example, Native Instruments’ FM8 can use around 1,000 parameters [?], and Teenage Engineering’s OP-1 synthesizer can be configured with up to 10^{76} parameters [?].

This incredibly large number of parameters is intended to give users fine-grained control over the generated audio, allowing for a wide variety of pitches (simulating a broad range of instruments and even non-instrument sounds, such as bird songs or car horns, among others). However, since the scope of this project is not focused on a commercial solution but rather on research purposes, a simple FM synthesizer will suffice.

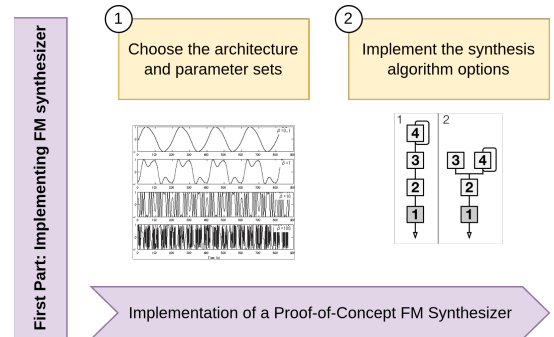


Fig. 1: Synthesis implementation and dataset generation.

Therefore, this work will implement a basic FM synthesizer with only 6 operators, inspired by the classic Yamaha DX7, which was one of the most disruptive milestones in the history of electronic music composition. In fact, this simple setup is enough to cover all FM synthesis theories and achieve good performance during AI model evaluation. In summary, the implemented FM Synthesizer (Figure ??) consists of 5 modulator operators, 1 carrier operator, 1 ADSR envelope, and 2 synthesis algorithms (one completely serial, and another with 3 serial operators followed by 2 parallel operators).

To summarize, the implemented synthesizer renders the audio signal according to the following steps:

figs/algs.png

Fig. 2: FM Synthesizer Algorithms.

- 1) Generate each operator signal by applying it to modulate the phase of the next operator in the sequence. In this implementation, only the phase is varied (similar to the Yamaha DX7), whereas a full FM synthesis would also be able to vary the frequency. This simplification helps to avoid complex noise-handling issues and promotes numerical stability. All signals are rendered at a 48 kHz sample rate.
- 2) Combine the output of each operator according to the synthesis algorithm (when there are parallel operators, they are summed).
- 3) Apply the ADSR envelope to the signal.
- 4) Downsample the signal from 48 kHz to 16 kHz (applying also a low-pass filter to avoid aliasing).

It is important to highlight that the high sample rate of 48 kHz was chosen to allow the FM synthesis to work well enough, since at high frequencies, there is a risk of obtaining aliasing and noise (due to the extremely high frequency components generated by the normal spacing of the FM operators). Furthermore, the low-pass filtering, used to downsample, is also useful to prevent the aliasing problem (after the downsampling process).

Finally, it is important to highlight that the entire synthesizer relies on 16 input parameters, consisting of the beta and ratio values for each modulator and for the carrier (totaling 12 parameters), plus 4 parameters for the ADSR envelope (attack, decay, sustain, and release times). As mentioned earlier, it is a very simple FM synthesizer, yet fully featured and sufficient for the purposes of the current research.

B. Generating the dataset

This work utilizes two datasets, each serving a distinct purpose: the generated dataset, referred to below as the synthetic dataset, which is used for training, and the NSynth

dataset, which is used to evaluate the model's generalization capabilities. However, in this section, only the first dataset will be considered, while the second will be addressed in Section ??.

One of the most challenging steps in the KDD process is obtaining and processing data. However, for the purposes of this work, an effective approach can be used. Similar to Claesson's approach [?], since the goal is to achieve good control over the synthesizer parameters, it is sufficient to generate many samples of synthesizer parameters and their respective audio outputs.

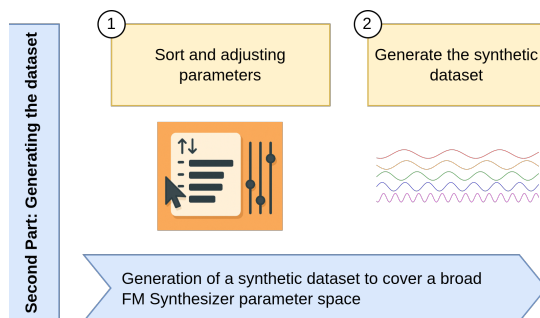


Fig. 3: Synthesis implementation and dataset generation.

The dataset used to train the model will consist of 5,000 samples generated by the implemented FM synthesizer, with the following characteristics:

- 5,000 samples.
- 4 seconds in duration.
- Monophonic samples (1-channel audio).
- 16 kHz audio sample rate (the synthesizer will generate samples at 48 kHz, but they will be downsampled to 16 kHz, to be compatible with the NSynth dataset).
- Pitches varying between 20 Hz and 6,000 Hz.
- Random ADSR envelopes.

The expectation is to achieve a level of learning where the model can accurately predict the parameters for a wide range of sounds that can be generated by the implemented FM synthesizer.

However, as the main purpose is to re-synthesize real instrument samples, the randomness of this dataset should be controlled to achieve reasonable music property samples, or, in other words, the dataset was constructed respecting the well-known limits and characteristics of real instruments:

- The minimum frequency for the carrier is 20 Hz, and the maximum is 6 kHz (for example, the maximum frequency for a violin is approximately 3 kHz; another important aspect for this maximum choice is that we need to stay under 8 kHz due to the output sample rate, because of the Nyquist frequency). The frequency is chosen by a uniform random distribution.
- The minimum ratio for each modulator is 1/8, and the maximum is 8 (to avoid aliasing in the final signal). Two methods are used to select the ratio: a) with 90% probability, the ratio will be a random uniform choice

TABLE I: Random Uniform ADSR Time Intervals

Parameter	Interval
Attack	[0.005, 0.5]
Decay	[0.02, 0.6]
Sustain	[0.1, 0.9]
Release	[0.05, 0.8]

between discretized values (for example: 1/8, 1/6, ..., 2, 3, 54, ..., 8; the idea is to simulate a real instrument); and b) with 10% probability, the ratio will be a uniform log distribution between 1/8 and 8 (allowing random metallic sounds, like bells, but respecting human auditory perception, where the difference between low frequencies is more perceived than between high frequencies).

- The minimum beta is 0, and the maximum is 8 (also to avoid aliasing in the final signal). The beta is chosen by a uniform random distribution, controlled such that 20% of the values are between 0 and 0.5, 60% are between 0.5 and 3, and 20% are between 3 and 8 (prioritizing rich timbres and avoiding pure sinusoidal sounds, as well as excessive metallic sounds).
- The amplitude is fixed at 1.0, to avoid different combinations of beta and amplitude generating the same frequency spectrum and the same timbre (prioritizing the CNN learning).

About the ADSR envelope, the following adjustments were made:

With these additional adjustments, the synthetic dataset will be more representative of simulating real instruments.

As each sample in the generated dataset is a complete 4-second sound (containing 64,000 audio samples, which corresponds to 16 kHz multiplied by 4 seconds), so a simple 75/25 train-test split was used, resulting in 3,750 audio files for training and 1,250 for testing.

But, from these 3,750 audio files, 20% of them will be used for validation, resulting in a total of 3,000 samples for training and 750 samples for testing.

C. Training the CNN

As described in Section ??, the main characteristic of the sound that should be reproduced to achieve the main work goal is the “timbre“, which is defined as the combination of frequencies present in the sound. It is clear that techniques such as filtering and Fourier Transform can be used to extract timbre components from an audio signal; however, they are insufficient for controlling an FM Synthesizer. In fact, the FM Synthesis control is, normally, an experimental process involving trial and error.

Additionally, each FM Synthesis operator generates additional frequency components in the signal, and there is no practical method to predict the correct parameter settings. However, as a mathematical function, it is easy to see that this synthesis pattern can be easily approximated using AI models.

Thus, the current work aims to use a CNN model, since it is a very effective method for learning patterns, respecting the internal data structure, and without requiring extensive pre-processing work. In other words, a CNN can simultaneously learn the time-dependent aspects of an audio signal and can act as a pattern extractor by applying a sequence of filtering operations, allowing the use of the raw audio format as input.

To train the CNN model, the already presented synthetic dataset will be used. Its main advantage is that it contains both the audio signal and the corresponding parameters used to generate each sample. In addition, this approach covers a wide range of the parameter space.

Consequently, the training process is straightforward and is expected to yield good performance.

Figure ?? presents the overall scheme of this training process.

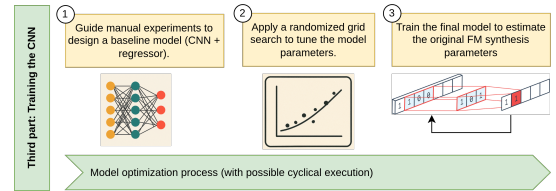


Fig. 4: Model training and optimization.

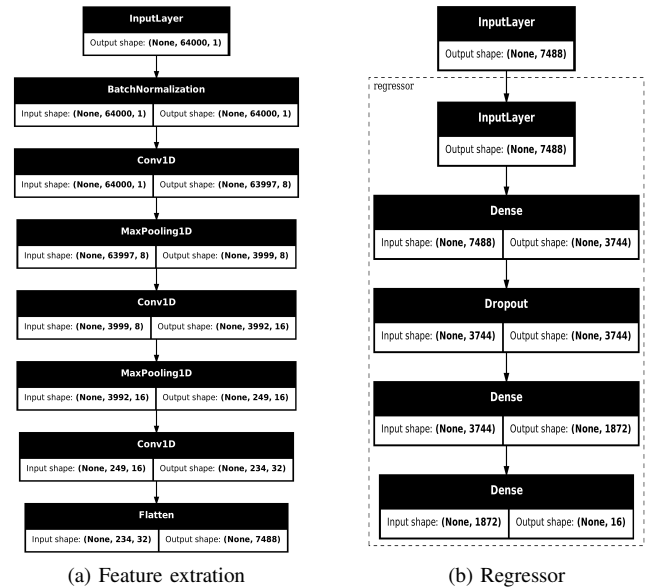


Fig. 5: Overview of the entire Model Architecture

The complete model is simply the sum of the two parts. The highlight the hyperparameters of each part are presented in Tables ?? and ??, respectively.

Other important highlights:

- The target variable is normalized to have a mean of 0 and a standard deviation of 1, prior to training.
- The Nadam optimizer was used.

TABLE II: Hyperparameters of Features Extraction (CNN part)

Component	Hyperparameter	Value
Conv1D (1)	Filters	8
	Kernel size	4
	Strides	1
MaxPooling1D (1)	Pool size	16
	Filters	16
Conv1D (2)	Kernel size	8
	Strides	1
	Pool size	16
MaxPooling1D (2)	Filters	32
	Kernel size	16
Conv1D (3)	Strides	1
	Pool size	16

TABLE III: Hyperparameters of the Regressor Network

Component	Hyperparameter	Value
Dense (1)	Activation	relu
Dropout	Rate	0.5
Dense (2)	Activation	relu
Dense (Output)	Activation	None
Optimizer	–	Nadam

- The Mean Squared Error (MSE) was used as the loss function.
- The recorded metrics were the Mean Absolute Error (MAE), and the Mean Squared Error (MSE).
- The model was trained for 200 epochs, with early stopping enabled using 20 epochs of patience and best weights restoring.
- The model was trained with a batch size of 32.

1) *Hyperparameters Optimization*: To reach the previously defined architecture, after an initial phase of experiments aimed at selecting a generic architecture able to achieve the proposed goals, a grid search process was applied to optimize the model’s hyperparameters.

This grid search did not differ substantially from the traditional process, but it is important to highlight the search space and the chosen strategy to obtain a good representation of the parameter space without requiring exhaustive experimentation. Table ?? shows the hyperparameters and their respective value options.

TABLE IV: Hyperparameters and Available Options

Hyperparameter	Options
Activation	gelu, swish, elu, relu, selu, silu
Bias (CNN)	True, False
Kernel Regularizer (CNN)	None, l1, l2
Bias (Regressor)	True, False
Dropout (Regressor)	0, 0.2, 0.3, 0.5
Kernel Regularizer (Regressor)	None, l1, l2
Optimizer	AdamW, RMSprop, Adam, Nadam, Adagrad, Adamax

However, calculating the total number of possible parameter combinations results in the remarkable value of 5,184 options. An exhaustive search over this space would take nearly nine days, given that 186 experiments required approximately seven hours.

To make the process viable, a random shuffle of the parameter search space was applied, producing a new order of experiments that did not require exhausting all options of a given parameter before moving on to the next.

Using this shuffled hyperparameter space, 186 experiments were conducted, representing nearly 3.6% of the total and requiring about seven hours of training.

This approach enabled the representation of a generic sample of the entire search space within a feasible time frame. Since any parameter option could appear in the sampled configurations, the random grid search allowed for faster and broader coverage of the parameter space.

Furthermore, each combination was used to train for 30 epochs with an early stopping patience of 5. All tested hyperparameters and their respective error metric values were recorded using the MLflow tool, which enabled more accurate comparisons among the top solutions. This, in turn, helped guide the selection of good hyperparameters and supported a more targeted, exhaustive training phase, already informed by the most successful configurations.

To provide a clearer understanding of this point, consider the experiment comparison graph presented in Figure ?? . Note that the following configurations appear successful, based on the three error metrics: RELU as activation function, CNN Bias equals to True, Regressor Bias equals to True, Regressor Dropout = 0.5, CNN Kernel Regularizer set to None, and Regressor Kernel Regularizer set to L2. It should be noted that the two best configurations, considering the MAE metric, were tested, and the second one (with respect to MAE) showed the best results without signs of overfitting.

D. Evaluating the Model

After training the CNN, a new dataset is used to evaluate the model’s ability to re-synthesize real instruments by calculating specific metrics focused on the FM synthesizer’s capacity to reproduce the same timbres.

The NSynth dataset, part of Google’s Magenta project, contains numerous real instrument samples, including both acoustic and electronic instruments, as well as some synthetic sounds generated by professional synthesizers. This dataset has a configuration similar to the one used for training, and its main purpose is to evaluate the model’s generalization capabilities, since it has never encountered it before.

Only the test set of the NSynth dataset will be used, with the following characteristics: 4,096 samples; 4 seconds in duration; Monophonic samples (1-channel audio); 16 kHz audio sample rate; 11 instrument families; pitches ranging from 27.5 Hz to 4,186.01 Hz; and the envelope includes a sustain of up to 3 seconds and a decay of 1 second.

The basic idea is to use the NSynth test samples as input to the model and, after predicting the synthesizer parameters,

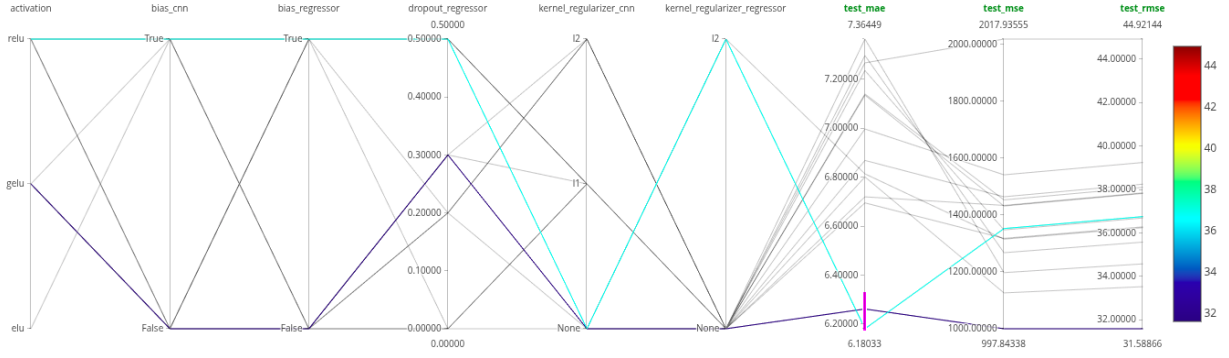


Fig. 6: MLflow grid search results showing the best error metrics (based on the test set).

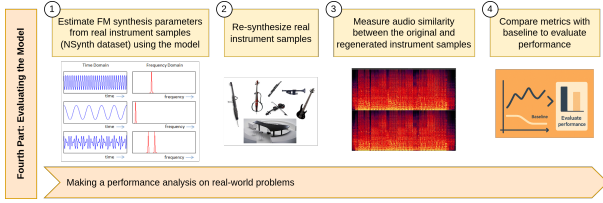


Fig. 7: Results evaluation.

employ the implemented synthesizer to re-synthesize all samples. Finally, audio comparison metrics are calculated between each input and its corresponding generated sound, based on the proposed evaluation measures:

- **FFT distance:** Used to compare the frequency components present in an audio signal.
- **STFT distance:** Used to compare the variation of frequency components over time.
- **Log-mel-spectrogram distance:** Used for a similar purpose as FFT distance, but typically models human cognitive pitch perception more accurately.

However, before using the predicted parameters as input to the synthesizer, a simple function was applied to approximate each ratio parameter to the nearest discrete ratio, according to the common behavior of musical harmonics (the same set of ratios used to randomize 90% of the generated dataset). Furthermore, some clipping operations were also applied to enforce the same limits imposed on the randomly generated dataset, such as maximum and minimum values for beta parameters.

To achieve a good evaluation, the proposed metrics were also used to compare each NSynth audio with a baseline consisting of a pure sinusoidal signal at 440 Hz (i.e., the A4 note, used as a reference to tune the majority of instruments).

V. EXPERIMENTAL RESULTS AND DISCUSSION

This section presents the final results of the model, including both training and audiocomparison metrics. The former is used to evaluate the model's ability to learn the entire parameter space, while the latter assesses its generalization capabilities, particularly its ability to emulate real instruments.

A. Training Results

The training and validation results are shown in Table ??, which summarizes the last five epochs of training.

TABLE V: Training and Validation Metrics by Epoch

Epoch	Loss	Train MAE	MSE	Validation Loss	MAE	MSE
70	0.603	0.527	0.545	0.622	0.517	0.562
71	0.601	0.526	0.542	0.637	0.520	0.569
72	0.605	0.526	0.545	0.621	0.516	0.565
73	0.602	0.524	0.541	0.625	0.522	0.568
74	0.611	0.528	0.548	0.625	0.520	0.563

Regarding the loss evolution across epochs, consider the following graphs:

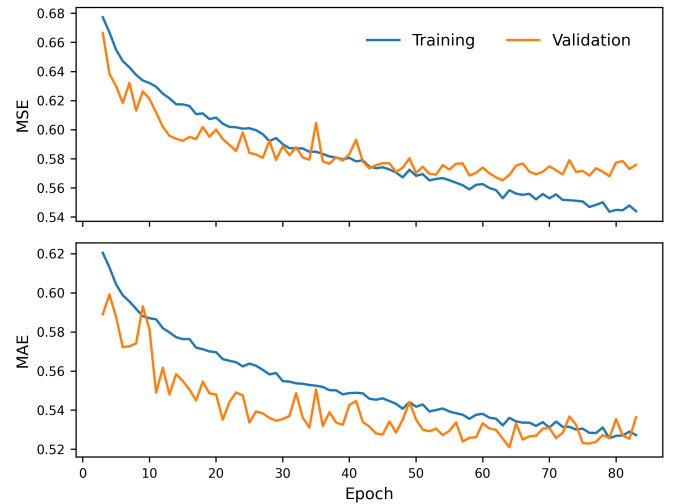


Fig. 8: Evolution of MAE and MSE during training.

After the training, still using a splitted part of the synthetic dataset, the test results was calculated. The predicted values were transformed back to the original space, and the results indicates:

- **Mean Absolute Error (MAE):** 6.488
- **Mean Squared Error (MSE):** 1410.097

- **Root Mean Squared Error (RMSE):** 37.551

And, these values demonstrate the model’s ability to reproduce the synthetic parameters.

B. Audio Comparison Results

As presented in Section ??, after training, the model’s generalization capabilities were tested through a process of re-synthesizing the NSynth dataset. The performance was evaluated by comparing the audio distance metrics between the original samples and the baseline samples with those between the re-synthesized samples and the original ones:

TABLE VI: Comparison between Resynthesis and Baseline with Improvement % (considering mean and standard deviation). [como você calculou o improvement??]

	FFT	STFT	Log Mel (raw)	Log Mel (normalized)
Resynthesis	12649.9 (6208.1)	2366.4 (1132.2)	138.73 (47.013)	0.0086 (0.0029)
Baseline	33508.8 (1887.36)	7290.4 (434.87)	184.87 (30.136)	0.0115 (0.0019)
Tuning (%)	62.270	67.540	24.960	24.930

Note that the Log Mel metric was compared, including a normalized version, which is obtained by dividing the raw Log Mel value by the product of the number of Mel bands and the time window quantity.

Furthermore, to facilitate a better understanding, the metrics were also compared by grouping the dataset by instrument family. This allows evaluation of the model’s performance across different sound classes. Table ?? shows the average values. We can note that the top three best results were obtained for keyboard, guitar, and mallet families.

TABLE VII: Average values by Instrument Group.

Group	FFT	STFT	Log Mel (raw)	Log Mel (normalized)
bass	11232.7	2144.32	135.610	0.00841
brass	14371.1	2682.95	167.617	0.01039
flute	14611.3	2872.69	141.819	0.00879
guitar	9692.0	1867.47	119.853	0.00743
keyboard	9619.3	1772.92	119.529	0.00741
mallet	8085.8	1632.45	120.610	0.00748
organ	18360.1	3453.13	166.433	0.01032
reed	12740.0	2267.08	140.701	0.00872
string	12883.7	2390.65	139.365	0.00864
vocal	31028.9	5271.26	212.646	0.01319

VI. CONCLUSION

This work investigated a technique designed to achieve two main objectives: re-synthesizing real musical instrument sounds and assessing the generalization ability of a model trained on randomized synthesizer parameters. The proposed convolutional neural network (CNN) performed effectively in both tasks, reproducing instrumental timbres with high fidelity

and generalizing well to unseen parameter configurations. Comparisons with NSynth samples confirmed the model’s ability to approximate realistic spectral characteristics across different instrument families. The stability of the normalized log-mel metric further indicates that the model does not overfit specific timbral structures, supporting its robustness and flexibility. Overall, the results validate the initial hypothesis that CNNs efficiently extract spectral–temporal representations from audio signals and that randomized dataset generation can successfully guide the learning of FM synthesis parameters.

Despite these positive outcomes, several limitations suggest directions for further research. The dataset’s focus on musical instruments restricts broader generalization, motivating future experiments with more diverse sound sources such as environmental and synthetic noises. The exclusive use of a linear ADSR envelope also limited the diversity of dynamic profiles, indicating the need for richer envelope variations. Furthermore, only two FM synthesis algorithms were explored, and satisfactory performance was achieved with a single configuration; investigating more complex operator structures may enhance synthesis versatility.

Future work should explore alternative neural architectures, such as autoencoders, to improve feature representation and generalization. Integrating CNN-based extraction with other strategies—such as STFT-based regression inputs or perceptually informed loss functions—may further enhance performance. AutoML techniques could also optimize the regression stage. Finally, retraining the system using a professional FM synthesizer implementation would provide a more realistic evaluation framework, advancing neural sound synthesis for both academic and creative applications.